

Hard link vs. soft link

Part A — Create the links

Create a directory named link_lab and navigate to it.

Create a file named security.txt containing the following text:

Linux File System Security Lab
Confidential Security Report

Create:

a hard link named security_hard.txt

a symbolic link named security_soft.txt

List all three files using a command that displays their inode numbers.

Record the inode numbers of:

security.txt

security_hard.txt

security_soft.txt

What difference do you observe?

```
(prathiksha@kali)~$ mkdir link_lab
(prathiksha@kali)~$ cd link_lab
(prathiksha@kali)~/link_lab$ pwd
/home/prathiksha/link_lab
(prathiksha@kali)~/link_lab$ echo -e "Linux File System Security Lab\nConfidential Security Report" > security.txt
(prathiksha@kali)~/link_lab$ cat security.txt
Linux File System Security Lab
Confidential Security Report
(prathiksha@kali)~/link_lab$ ln security.txt security_hard.txt
(prathiksha@kali)~/link_lab$ ln -s security.txt security_soft.txt
(prathiksha@kali)~/link_lab$ ls -li
total 8
1078993 -rw-rw-r-- 2 prathiksha prathiksha 60 Sep  2 22:15 security_hard.txt
1078629 lrwxrwxrwx 1 prathiksha prathiksha 12 Sep  2 22:16 security_soft.txt -> security.txt
1078993 -rw-rw-r-- 2 prathiksha prathiksha 60 Sep  2 22:15 security.txt
(prathiksha@kali)~/link_lab$ ls -li security.txt security_hard.txt security_soft.txt
1078993 -rw-rw-r-- 2 prathiksha prathiksha 60 Sep  2 22:15 security_hard.txt
1078629 lrwxrwxrwx 1 prathiksha prathiksha 12 Sep  2 22:16 security_soft.txt -> security.txt
1078993 -rw-rw-r-- 2 prathiksha prathiksha 60 Sep  2 22:15 security.txt
(prathiksha@kali)~/link_lab$ cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
(prathiksha@kali)~/link_lab$ cat security_soft.txt
Linux File System Security Lab
Confidential Security Report
```

Observation:

security.txt and security_hard.txt have the same inode number because a hard link points directly to the same file data.

security_soft.txt has a different inode number because a symbolic (soft) link is a separate file that points to the path of security.txt.

Part B — Examine File Metadata

Use ls -l to examine the three files. Identify the file type of each file from the first character of the permission field. Compare the link count of the original file, hard link, and symbolic link. Explain the observed values.

Compare the file sizes of the original file, hard link, and symbolic link. Why might the size of the symbolic link differ from the original file?

```

(prathiksha@kali)~/link_lab
$ ls -l security.txt security_hard.txt security_soft.txt
-rw-rw-r-- 2 prathiksha prathiksha 60 Sep  2 22:15 security_hard.txt
lrwxrwxrwx 1 prathiksha prathiksha 12 Sep  2 22:16 security_soft.txt -> security.txt
-rw-rw-r-- 2 prathiksha prathiksha 60 Sep  2 22:15 security.txt

(prathiksha@kali)~/link_lab
$ stat security.txt
File: security.txt
Size: 60          Blocks: 8          IO Block: 4096   regular file
Device: 8,1      Inode: 1078993   Links: 2
Access: (0664/-rw-rw-r-- )  Uid: ( 1000/prathiksha)   Gid: ( 1000/prathiksha)
Access: 2026-09-02 22:16:47.976098495 +0530
Modify: 2026-09-02 22:15:44.847710326 +0530
Change: 2026-09-02 22:16:05.429859800 +0530
Birth: 2026-09-02 22:15:44.847710326 +0530

(prathiksha@kali)~/link_lab
$ stat security_hard.txt
File: security_hard.txt
Size: 60          Blocks: 8          IO Block: 4096   regular file
Device: 8,1      Inode: 1078993   Links: 2
Access: (0664/-rw-rw-r-- )  Uid: ( 1000/prathiksha)   Gid: ( 1000/prathiksha)
Access: 2026-09-02 22:16:47.976098495 +0530
Modify: 2026-09-02 22:15:44.847710326 +0530
Change: 2026-09-02 22:16:05.429859800 +0530
Birth: 2026-09-02 22:15:44.847710326 +0530

(prathiksha@kali)~/link_lab
$ stat security_soft.txt
File: security_soft.txt -> security.txt
Size: 12          Blocks: 0          IO Block: 4096   symbolic link
Device: 8,1      Inode: 1078629   Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/prathiksha)   Gid: ( 1000/prathiksha)
Access: 2026-09-02 22:16:23.006854179 +0530
Modify: 2026-09-02 22:16:14.628300447 +0530
Change: 2026-09-02 22:16:14.628300447 +0530
Birth: 2026-09-02 22:16:14.628300447 +0530

```

File type — first character of permission field

File	First char	Type
security.txt	-	Regular file
security_hard.txt	-	Regular file
security_soft.txt	l	Symbolic link

Compare link counts

File	Link Count
security.txt	2
security_hard.txt	2
security_soft.txt	1

Explanation: security.txt and security_hard.txt both show link count 2 because a hard link doesn't create a new file — it's just another name pointing to the *same inode* (1078993). The filesystem counts how many filenames reference that inode, so creating the hard link bumped the count from 1 to 2 for both entries.

security_soft.txt shows link count 1 because a symbolic link is a completely separate file with its own inode (1078629). It doesn't add a reference to the target's inode at all — it just stores the target's path as its content.

Use the stat command on all three files and record the following information

Parameter	Original	Hard Link	Soft Link
Inode number	1078993	1078993	1078629
File type	regular file	regular file	symbolic link
Size	60	60	12

Link count	2	2	1
Permissions	0664 (rw-rw-r--)	0664 (rw-rw-r--)	0777 (lrwxrwxrwx)

Why might the size of the symbolic link differ from the original file?

The symlink's size (12 bytes) is simply the length of the string `security.txt` — the path it points to. It does not store or mirror the target's actual data.

The hard link's size (60 bytes) matches the original exactly because it isn't a separate file at all — it shares the same inode (1078993) as `security.txt`, meaning it points to the exact same data blocks on disk. Same inode = same size, same content, same everything except the filename.

Part C — Investigate readlink

Execute `readlink` on `security_soft.txt`. What information is displayed?

`readlink` prints the target path stored inside the symlink — i.e., the string the symlink points to. This is exactly the content that was recorded when the symlink was created with `ln -s`. It does not resolve or verify whether the target actually exists; it just reads the link's stored value.

Execute `readlink` on `security_hard.txt`. What do you observe?

`readlink` produces no output for the hard link. Some versions may just return silently with a non-zero exit status; it doesn't print `security.txt` or anything else.

Based on your observations, explain why `readlink` works differently for hard links and symbolic links.

- A hard link is just another name for the same file (same inode). There's no target path stored anywhere, so `readlink` has nothing to show.
- A symbolic link is a separate tiny file that only stores a path pointing to another file. `readlink` reads that stored path and prints it.

That's why `readlink` works on symlinks but returns nothing for hard links — one has a path to read, the other doesn't.

Use:

```
readlink -f security_soft.txt
```

What does the `-f` option reveal?

`-f` makes `readlink` follow the symlink to its final target.

It shows the target's complete absolute path.

It also resolves `.`, `..`, and other symlinks.

So, it shows the file's exact location on the system.

```
(prathiksha@kali)~[~/link_lab]
$ readlink security_soft.txt
security.txt

(prathiksha@kali)~[~/link_lab]
$ readlink security_hard.txt

(prathiksha@kali)~[~/link_lab]
$ readlink -f security_soft.txt
/home/prathiksha/link_lab/security.txt
```

Part D — Content Modification

Append the following line to `security.txt`:

Unauthorized login detected

Display the contents using all three filenames.

Is the newly added content visible through both links? Explain why.

Yes — visible through both. `security_hard.txt` shares the exact same inode as `security.txt`, so they point to the identical data blocks on disk; any change to one is instantly reflected in the other.

`security_soft.txt` doesn't store any data of its own — it just redirects to `security.txt`, so reading it always shows whatever `security.txt` currently contains

Modify the file using `security_hard.txt`. Check the contents through `security.txt` and `security_soft.txt`. What do you observe?

Observation: The new line appears in both `security.txt` and `security_soft.txt` too. Since hard links share the same inode, editing through any linked name updates the single underlying file — there's really only one copy of data, accessible via multiple names.

```
(prathiksha@kali)~[/link_lab]
$ echo "Unauthorized login detected" >> security.txt

(prathiksha@kali)~[/link_lab]
$ cat security.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected

(prathiksha@kali)~[/link_lab]
$ cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected

(prathiksha@kali)~[/link_lab]
$ cat security_soft.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected

(prathiksha@kali)~[/link_lab]
$ echo "Second unauthorized attempt" >> security_hard.txt

(prathiksha@kali)~[/link_lab]
$ cat security.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Second unauthorized attempt

(prathiksha@kali)~[/link_lab]
$ cat security_soft.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Second unauthorized attempt
```

Part E — Rename Experiment

Rename:

`security.txt`

to:

`incident.txt`

Try accessing the content through `security_hard.txt`. Does it still work?

The hard link doesn't reference the *name* `security.txt` — it references the *inode*. Renaming only changes the directory entry name, not the inode, so the hard link is unaffected.

Try accessing the content through `security_soft.txt`. Does it still work?

error like `cat: security_soft.txt: No such file or directory`. The symlink stores the literal path string `security.txt`, which no longer exists after the rename.

Use `readlink security_soft.txt` after renaming the original file. What path is still stored in the symbolic link?

Explain why renaming the original file affects the symbolic link differently from the hard link.

A hard link points to the inode (the actual data), so it doesn't care what name the file has — renaming doesn't touch the inode at all. A symbolic link points to a name/path, not the inode, so once that name is gone, the symlink becomes "dangling" — it still holds the old path but that path no longer resolves to anything.

```
(prathiksha@kali)~[/link_lab]
$ mv security.txt incident.txt

(prathiksha@kali)~[/link_lab]
$ cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Second unauthorized attempt

(prathiksha@kali)~[/link_lab]
$ cat security_soft.txt
cat: security_soft.txt: No such file or directory

(prathiksha@kali)~[/link_lab]
$ readlink security_soft.txt
security.txt
```

Part F — Deletion Experiment

Recreate the links if necessary and then perform the following.

Delete the original file.

Check whether security_hard.txt can still be accessed.

Check whether security_soft.txt can still be accessed.

Use ls -l and readlink to investigate the symbolic link after deleting its target.

What is a dangling/broken symbolic link?

A symlink whose target path no longer exists (deleted, moved, or renamed). The symlink file itself still exists and still stores the old path, but any attempt to open/read/write through it fails because the destination is gone.

Does deleting the original filename immediately delete the underlying data when a hard link exists? Justify your answer using the link count and inode.

No. Deletion (rm) just removes one directory entry and decrements the inode's link count by 1. The actual data blocks are only freed when the link count drops to 0 (and no process has the file open). Since security_hard.txt still references the same inode, the link count was 2 before deletion and becomes 1 after — the inode is still "alive," so the data remains fully accessible through security_hard.txt.

```
(prathiksha@kali)~/link_lab
$ ln incident.txt security_hard.txt
ln: failed to create hard link 'security_hard.txt': File exists

(prathiksha@kali)~/link_lab
$ ln -s incident.txt security_soft.txt
ln: failed to create symbolic link 'security_soft.txt': File exists

(prathiksha@kali)~/link_lab
$ rm incident.txt

(prathiksha@kali)~/link_lab
$ cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Second unauthorized attempt

(prathiksha@kali)~/link_lab
$ cat security_soft.txt
cat: security_soft.txt: No such file or directory

(prathiksha@kali)~/link_lab
$ ls -l security_soft.txt
lrwxrwxrwx 1 prathiksha prathiksha 12 Sep  2 22:16 security_soft.txt -> security.txt

(prathiksha@kali)~/link_lab
$ readlink security_soft.txt
security.txt
```

Part G — Directory and Filesystem Behaviour

1. Create a directory named evidence. Attempt to create a hard link to this directory. Record the result.

mkdir evidence

In evidence evidence_hard

2. Create a symbolic link named evidence_link pointing to the evidence directory. Was the operation successful?

In -s evidence evidence_link

Check using:

ls -l

3. Based on your experiment, compare hard and symbolic links with respect to directory linking.

A hard link to a directory could not be created, whereas a symbolic link to the directory was created successfully. Therefore, symbolic links can point to directories, while normal hard links to directories are not permitted.

4. If multiple filesystems/mount points are available, attempt to create a hard link across filesystems.

Check the filesystems:

df -T

Command:

```
ln /filesystem1/file.txt /filesystem2/file_hard
```

Observation: The hard link cannot be created across different filesystems and produces an Invalid cross-device link error.

5. Attempt to create a symbolic link across filesystems.

```
ln -s /filesystem1/file.txt /filesystem2/file_symbolic
```

Observation: The symbolic link can be created across different filesystems successfully.

Final Comparison

Parameter	Hard Link	Symbolic Link
Inode number	Same as target	Different from target
File type	Regular file	Symbolic link
Link count	Increases for target	Target link count unchanged
File size	Same as target	Size of stored pathname
Stores target pathname?	No	Yes
readlink output	No meaningful output	Displays target pathname
Effect of modifying target	Changes are reflected	Changes are reflected
Effect of renaming target	Still works	May become dangling
Effect of deleting target	Still works	Becomes dangling/broken
Can become a dangling or broken link?	No	Yes
Directory linking	Not allowed for normal users	Allowed
Cross-filesystem linking	Not possible	Possible

sort

1. Suppose alerts.txt contains:

```
192.168.1.10 admin 5
10.0.0.8 root 12
172.16.0.5 guest 2
10.0.0.20 admin 8
```

Format: IP-address username failed attempts

2. Sort according to failed attempts.

```
sort -k3,3n alerts.txt
```

Output:

```
172.16.0.5 guest 2
192.168.1.10 admin 5
10.0.0.20 admin 8
10.0.0.8 root 12
```

3. Sort a list of suspicious IP addresses alphabetically.

```
sort suspicious_ips.txt
```

4. Display only unique suspicious IP addresses using sort.

```
sort -u suspicious_ips.txt
```

5. Sort a list of detected port numbers numerically.

```
sort -n ports.txt
```

6. Display port numbers in descending numerical order.

```
sort -nr ports.txt
```

7. Sort usernames without considering uppercase/lowercase differences.

```
sort -f users.txt
```

8. From a failed-login log, count how many times each IP address occurs and display the most frequent IP first.

```
awk '{print $1}' failed.log | sort | uniq -c | sort -nr
```

9. Given IP, username, failed_attempts, sort records based on the number of failed attempts.

```
sort -t',' -k3,3n login.csv
```

10. Sort a CSV vulnerability report based on severity score.

```
sort -t',' -k2,2n vulnerabilities.csv
```

For highest severity first:

```
sort -t',' -k2,2nr vulnerabilities.csv
```

11. Extract IP addresses using grep, then use sort and uniq to identify unique IP addresses.

```
grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' failed.log | sort | uniq
```

12. Create a pipeline that displays the top 5 IP addresses responsible for the most failed login attempts.

```
awk '{print $1}' failed.log | sort | uniq -c | sort -nr | head -5
```